HPC/hpc-1.cpp

```cpp
/*
Design and implement Parallel Breadth First Search and Depth First Search
based on existing algorithms using OpenMP. Use a Tree or an undirected
graph for BFS and DFS .
Use large input data (>1000)
Compare parallel and sequential execution.
*/

#include <omp.h>
#include <iostream>
#include <vector>
#include <queue>
#include <chrono>
using namespace std;
using namespace chrono;

// Serial DFS Implementation
class SerialDFS {
public:
    void recursiveDfs(int node, vector<int> adj[], int visited[], vector<int> &res) {
        visited[node] = 1;
        res.push_back(node);
        for (auto it : adj[node]) {
            if (!visited[it]) {
                recursiveDfs(it, adj, visited, res);
            }
        }
    }

    vector<int> dfs(int n, vector<int> adj[]) {
        int visited[n];
        for (int i = 0; i < n; i++) visited[i] = 0;
        vector<int> res;
        recursiveDfs(0, adj, visited, res);
        return res;
    }
};

// Parallel DFS Implementation
class ParallelDFS {
public:
    void recursiveDfs(int node, vector<int> adj[], int visited[], vector<int> &res) {
        visited[node] = 1;
        res.push_back(node);

        #pragma omp parallel for
        for (size_t i = 0; i < adj[node].size(); i++) {
            int neighbor = adj[node][i];
            if (!visited[neighbor]) {
                #pragma omp task
```

```
                recursiveDfs(neighbor, adj, visited, res);
            }
        }
    }

    vector<int> dfs(int n, vector<int> adj[]) {
        int visited[n];
        for (int i = 0; i < n; i++) visited[i] = 0;
        vector<int> res;

        #pragma omp parallel
        {
            #pragma omp single
            {
                recursiveDfs(0, adj, visited, res);
            }
        }
        return res;
    }
};

// Serial BFS Implementation
class SerialBFS {
public:
    vector<int> bfs(int n, vector<int> adj[]) {
        vector<int> visited(n, 0);
        visited[0] = 1;
        queue<int> q;
        q.push(0);
        vector<int> res;
        while (!q.empty()) {
            int node = q.front();
            q.pop();
            res.push_back(node);
            for (auto it : adj[node]) {
                if (!visited[it]) {
                    visited[it] = 1;
                    q.push(it);
                }
            }
        }
        return res;
    }
};

// Parallel BFS Implementation
class ParallelBFS {
public:
    vector<int> bfs(int n, vector<int> adj[]) {
        vector<int> visited(n, 0);
        visited[0] = 1;
        vector<int> res;
        queue<int> q;
        q.push(0);
```

```

        while (!q.empty()) {
            vector<int> currentLevel;

            while (!q.empty()) {
                int node = q.front();
                q.pop();
                res.push_back(node);
                currentLevel.push_back(node);
            }

            #pragma omp parallel for
            for (size_t i = 0; i < currentLevel.size(); i++) {
                int node = currentLevel[i];
                for (auto neighbor : adj[node]) {
                    if (!visited[neighbor]) {
                        #pragma omp critical
                        {
                            if (!visited[neighbor]) {
                                visited[neighbor] = 1;
                                q.push(neighbor);
                            }
                        }
                    }
                }
            }
        }
        return res;
    }
};

// Commenting out the hardcoded graph
    // adj[0] = {1, 2};
    // adj[1] = {0, 3};
    // adj[2] = {0, 4};
    // adj[3] = {1};
    // adj[4] = {2};

    // Random graph generator
    vector<int> *generateRandomGraph(int n, int m) {
        vector<int> *adj = new vector<int>[n];
        srand(time(0)); // Seed for random number generation

        for (int i = 0; i < m; i++) {
            int u = rand() % n; // Random node u
            int v = rand() % n; // Random node v
            if (u != v) { // Avoid self-loops
                adj[u].push_back(v);
                adj[v].push_back(u); // Since the graph is undirected
            }
        }
        return adj;
    }

```

```
    // Generate a large graph
    int n = 1000; // Number of nodes
    int m = 5000; // Number of edges
    vector<int> *adj = generateRandomGraph(n, m);

int main() {
    SerialBFS serialBfs;
    ParallelBFS parallelBfs;
    SerialDFS serialDfs;
    ParallelDFS parallelDfs;

    // Measure Serial BFS runtime
    auto start_serial_bfs = high_resolution_clock::now();
    vector<int> res_serial_bfs = serialBfs.bfs(n, adj);
    auto end_serial_bfs = high_resolution_clock::now();
    double serial_bfs_time = duration<double>(end_serial_bfs - start_serial_bfs).count();

    // Measure Parallel BFS runtime
    auto start_parallel_bfs = high_resolution_clock::now();
    vector<int> res_parallel_bfs = parallelBfs.bfs(n, adj);
    auto end_parallel_bfs = high_resolution_clock::now();
    double parallel_bfs_time = duration<double>(end_parallel_bfs - start_parallel_bfs).count();

    // Measure Serial DFS runtime
    auto start_serial_dfs = high_resolution_clock::now();
    vector<int> res_serial_dfs = serialDfs.dfs(n, adj);
    auto end_serial_dfs = high_resolution_clock::now();
    double serial_dfs_time = duration<double>(end_serial_dfs - start_serial_dfs).count();

    // Measure Parallel DFS runtime
    auto start_parallel_dfs = high_resolution_clock::now();
    vector<int> res_parallel_dfs = parallelDfs.dfs(n, adj);
    auto end_parallel_dfs = high_resolution_clock::now();
    double parallel_dfs_time = duration<double>(end_parallel_dfs - start_parallel_dfs).count();

    // Print results
    cout << "Serial BFS" << endl;
    for (int node : res_serial_bfs)
        cout << node << " ";
    cout << '\n';

    cout << "Parallel BFS" << endl;
    for (int node : res_parallel_bfs)
        cout << node << " ";
    cout << '\n';

    cout << "Serial DFS" << endl;
    for (int node : res_serial_dfs)
        cout << node << " ";
    cout << '\n';
```

```

    cout << "Parallel DFS" << endl;
    for (int node : res_parallel_dfs)
        cout << node << " ";
    cout << '\n';

    // Print timings
    cout << "Serial BFS Runtime: " << serial_bfs_time << " seconds" << endl;
    cout << "Parallel BFS Runtime: " << parallel_bfs_time << " seconds" << endl;
    cout << "Serial DFS Runtime: " << serial_dfs_time << " seconds" << endl;
    cout << "Parallel DFS Runtime: " << parallel_dfs_time << " seconds" << endl;

    return 0;
}

/*
Output:

Serial BFS Runtime: 0.000608666 seconds
Parallel BFS Runtime: 0.00610525 seconds
Serial DFS Runtime: 0.000306125 seconds
Parallel DFS Runtime: 0.001319 seconds

*/
```